How Long May a Reader Hold the Lock? A Controlled Read-Cost Study of Coarse Locking, Descriptor Leasing, and Immutable Snapshots in an LSM Engine

V V Ramarathinam Iyer
Department of Database Systems, School of Computer Science and Engineering
Vellore Institute of Technology, Vellore, Tamil Nadu, India
Email: ram.iyer200722@gmail.com

***Abstract*—We evaluated coarse locking (A), descriptor leasing (A-min), and immutable reader-set snapshots (B) in a Go LSM engine in a pre-registered 519-trial campaign. All 519 trials succeeded on the first attempt. Across ten matched triplets per cell, A's write-throughput ratio to A-min decreased with injected read cost (Page's $L = 294$, exact one-sided $p = 7.86 \times 10^{-9}$ at both reader counts). At four readers, the ratio was 0.988 without delay, 0.758 at 200 μs, and 0.361 at 2 ms. Unexpectedly, A completed more reads than both alternatives through 200 μs; A/A-min read ratios were 1.19–1.45 at four readers and 1.09–1.43 at sixteen. Exploratory counters showed approximately two reads per reader per commit under A, consistent with two exclusive acquisitions and Go's reader-admission policy. A separately pre-registered 160-trial intervention removed the second acquisition below threshold: A's reads per reader per commit fell from about 2.00 to 1.00 in all eight cells, and its low-cost read advantage was no longer detectable. Read throughput also fell in both arms, so the second acquisition was not simply waste. A-min and B met the throughput equivalence criterion in all ten write and fourteen read cells; publication lock-wait equivalence was not established. Churn did not isolate snapshot lifecycle cost. Findings are bounded by this engine and workload, including exclusive writer conflict validation and WAL synchronization.**

***Index Terms*—LSM tree, reader-writer lock, descriptor leasing, immutable snapshots, controlled experiment.**

I. Introduction

LSM trees buffer writes in memory and persist them as sorted runs [1]. Point readers must access those files safely while flushes and compactions change the set of files. Holding a shared manager lock across a lookup gives a simple lifetime guarantee but delays anyone who needs the lock exclusively; capturing stable file references first separates file lifetime from I/O duration. How much this matters depends on how long the lock is held, on the commit path, and on the lock's admission policy.

We compared three approaches in one engine: A holds the read lock across SSTable I/O, A-min leases individual table readers, and B pins an immutable reader-set descriptor. We varied read cost with an injected delay before block reads and kept the engine's defective earlier reader as a separate historical operating point. We asked how A's write penalty changes with read cost (RQ1), whether leasing matches snapshots within a declared margin (RQ2), and how the approaches compare under reader-set churn (RQ3).

This paper makes the following contributions:

- A pre-registered, paired, dose-controlled comparison across 17 cells and 519 trials, spanning measured read critical sections from about 14 μs to 430 ms under A.
- A write-penalty trend that was small at tens of microseconds and severe at milliseconds, and an unpredicted read-throughput crossover.
- An exploratory account relating A's read throughput to exclusive acquisitions per commit and Go's reader admission, with reads per reader per commit near 2.00 in every non-legacy cell, then tested by a separately pre-registered 160-trial intervention.
- Bounded evidence on leasing versus snapshots: throughput equivalence within 10% in all tested X1/X2 cells, no established publication lock-wait equivalence, and correction of an earlier attribution of churn cost to snapshot lifecycle.

The intervention supported the account while exposing a read/write trade-off; neither campaign establishes that snapshots are unnecessary in other engines.

II. Background and Earlier Measurements

Luo and Carey survey LSM buffering and compaction [5]. LevelDB and RocksDB provide production reference points [2], [3]; our arms were implementations in a smaller engine, not measurements of those systems.

The earlier SSTable reader scanned from the target block toward the file's data end, with about four unbuffered syscalls per record and no early stop after passing the target key. This was a defect, now retained only as `legacy_scan`. A 960-trial WSL2 campaign comparing A and B and a Windows campaign truncated by a stopping rule (60 of 330 planned trials, 18 complete triplets) used it. They reported B completing 3.3–18.5 times A's writes, and churn throughput B/A of 0.63–0.74 at $r \leq 4$. These runs motivated this study; they were not pooled with it and no inference was carried

TABLE I
SYNCHRONIZATION ARMS IN THE MAIN CAMPAIGN (DEFAULT COMMIT PATH), AUDITED AGAINST THE IMPLEMENTATION.

Property	A	A-min	B
Memory-tier search under `Mu.RLock`	yes	yes	yes
Reader SSTable I/O under `Mu.RLock`	yes	no	no
Disk-reader lifetime	lock	table leases	set pin
Per-read lifetime synchronization	`Mu`	atomic leases	`snapshotMu`
New set descriptor on publication	no	no	yes
Conflict check and WAL sync under `Mu.Lock`	yes	yes	yes
Exclusive acquisitions per successful commit	2	2	2

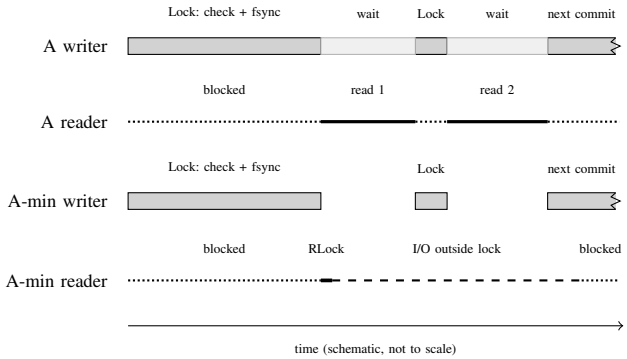


Fig. 1. One commit cycle under A and A-min (schematic, not a measured trace). Under A the writer waits for each read and each reader completes one read per exclusive acquisition. Under A-min the reader's I/O is outside the lock and can outlast the short second acquisition. A: two reads per commit; A-min: fewer.

forward. This study supersedes the earlier claims, including the withdrawn $\pm 5\%$ equivalence claim and any claim about A-min under churn.

III. Engine and Synchronization Arms

A. Storage and point reads

The memtable used a skip list [11], promotion probability 0.5 and maximum height 32. Internal keys appended an eight-byte big-endian, bitwise-inverted encoding of $(\text{seq} \ll 8) \,|\, \text{delete}$ to the user key: 56 effective sequence bits, with the low bit indicating a tombstone. Newer versions sorted first within a user key. Frozen memtables flushed into SSTables.

Data blocks used a 4 KiB flush threshold; a whole record could take a block beyond it. A flat sparse index stored block first keys and offsets, with an index-location footer. The corrected reader checked the cached per-table user-key range, binary-searched the index, and used one `ReadAt` per visited block. It stopped at a visible version or greater user key. Versions could straddle blocks, requiring further block reads. Open-time range-metadata reads bypassed delay. The path had no application block cache.

B. Lifetime management and commit path

All arms searched active and frozen memory under `Mu.RLock`; A retained it through SSTable I/O. A-min copied reader pointers into reusable scratch storage and took atomic per-table leases under that lock, searched outside `Mu`, then released leases. Shutdown closed admission and drained active and retired leases; timeout returned an error and preserved leased descriptors.

B pinned the reader set under the short manager read lock and `snapshotMu`, then searched outside both. Release acquired `snapshotMu`. Publication allocated a set, assigned `currentSnapshot` under mutexes, and retired the old set. Drained sets released constituent references; eligible closure/unlinking ran outside both mutexes. This was mutex-coordinated, not a lock-free pointer swap (Table I).

`Transaction.Commit` acquired `Mu.Lock`, validated conflicts via `latestValueLocked`, appended and synchronized the WAL, and updated memory. It unlocked, then called `checkAndTriggerFlush`, which reacquired the exclusive lock even without a flush. Each successful main-campaign commit therefore made two exclusive acquisitions. Only the first wait was instrumented. Conflict validation could read delayed SSTable blocks under the exclusive lock in all arms. A-min and B shortened the reader's hold, not the writer's.

IV. Experimental Design

A. Host, workload, and controlled dose

The host was an AMD Ryzen 7 7445HS (6 cores, 12 logical CPUs), 15.2 GiB RAM, and Samsung MZVL8512HFLU NVMe SSD, running Windows 11 Home build 26200 on NTFS. The runner used Go 1.26.3 and `GOMAXPROCS=12`. AC power, the Best performance overlay on the Balanced base plan, and enabled Defender real-time protection were retained. Timing used QueryPerformanceCounter.

Fixtures had 10,000 keys, four disjoint SSTables of 2,500 keys, and 128-byte values. Closed-loop uniform reads targeted 0.95 hit ratio. Trials used 2 s warmup, 10 s measurement, 200 ms cooldown, and synchronization instrumentation. WAL fsync was enabled during measurement and disabled during fixture construction.

The 17 cells comprised X1: $r \in \{4, 16\}$, $d \in \{0, 20, 200\}$ μs, 2 ms, or legacy (ten cells), one free-running writer, 1 MiB memtable; X2: $r = 16$, $d = 0$, 200 μs, 2 ms, or legacy (four cells), no writer, twenty pre-frozen memtables released for physical publication every 0.5 s from 0.25 to 9.75 s; X3: $r \in \{1, 4, 16\}$, $d = 0$, one writer, 16 KiB memtable (three cells). Legacy used the retained scan without added delay.

Delay ran immediately before each point-read `ReadAt`, including conflict-validation reads. Sub-millisecond doses used a CPU-yielding spin; 2 ms used sleep then spin for the final 0.5 ms. The earlier single-goroutine probe measured approximately 9 μs, 35 μs, 215 μs, and 2.25 ms per read. These probe costs differed from concurrent lock holds, reported separately.

B. Pairing, scope, and analysis

The protocol was frozen in git before non-smoke trials. Ten triplets per cell paired arms on seed and operation-index streams; completion-dependent prefixes could differ.

Seeds were the first eight SHA-256 bytes of decimal master 20261030, cell ID, and triplet index separated by vertical bars, interpreted as big-endian int64. Ten rounds randomized cells using master plus round; arms cycled through six permutations. Same-seed anchors ran at start, after round five, and end: 510 cell trials plus nine anchors.

W and R counted only in-window write and read completions. Latency histograms recorded all in-window completions without sample drops; late completions were separate. Synchronization counters reset at window start and included post-window drain, excluding fixture construction and warmup. Flush diagnostics also included drain and were not strictly in-window counts.

H1 used one-sided exact Page testing [13] on $S = W_A/W_{\text{A-min}}$ across non-legacy doses, blocking by triplet. H2 used TOST [14] on paired $\ln(\text{A-min}/B)$, 90% paired-t CIs, margin $\pm\ln(1/0.90)$ (rounded ratio $[0.90, 1.11]$), and Holm correction [15] within W (ten cells), R (fourteen), and publication lock-wait p99 (four). Adjusted $p < 0.05$ established equivalence; a CI wholly beyond a margin meant not equivalent; otherwise inconclusive. H3 used two-sided 95% paired-t CIs, without directional prediction; descriptive ratios also had 95% CIs. Bootstrap sensitivity used 10,000 percentile resamples, seed 20261030. Cells were not pooled; no planned write-zero adjustment was needed.

V. Results

A. RQ1: dose-response

All 519 trials succeeded on the first attempt: no retries, exclusions, or zero adjustments. Execution ran on 2026-11-02, 12:41–14:35 UTC, about 1 h 54 min. Table II reports rates, paired ratios, and A's measured critical sections; Fig. 2 shows writes. All pairwise ratios with 95% CIs for every cell are in the analysis output in the repository.

At both reader counts, Page's $L = 294$, exact one-sided $p = 7.86 \times 10^{-9}$, $n = 10$. Across non-legacy doses, geometric S decreased from 0.988 to 0.361 at $r = 4$ and 0.937 to 0.311 at $r = 16$ (Table II). Legacy ratios were 0.0042 and 0.0017. The trend did not distinguish all adjacent doses: at $r = 16$, zero and 20 μs had mean ranks 3.4 and 3.6, without descriptive separation.

Mean A read critical sections ranged from 13.6 μs to 125 ms at $r = 4$ and 21.2 μs to 430 ms at $r = 16$; Table II gives every measured dose. A-min/B cell means remained 1.5–4.9 μs. Nominal delay differed from measured hold.

Read throughput showed an unpredicted result (Fig. 3). At nominal $d = 0$, 20 μs, 200 μs, and 2 ms, A/A-min read ratios were 1.193, 1.452, 1.187, and 0.725 at $r = 4$, and 1.097, 1.432, 1.093, and 0.625 at $r = 16$. Legacy ratios were 0.489 and 0.522, respectively. A completed more reads than A-min and B through 200 μs, crossing over between 200 μs and 2 ms. At low injected cost, coarse locking gave higher read throughput at a modest write cost in this engine and workload; by 200 μs the write cost was larger. This read-side pattern was descriptive, neither predicted nor a pre-registered test.

B. Lock-admission mechanism (exploratory, post hoc)

Post-data source review identified two exclusive acquisitions per commit. Go's RWMutex blocks new readers behind a waiting writer and releases queued readers on unlock [12]. With a writer continually waiting, this admission alternates reader and writer phases and resembles the phase-fair behaviour described by Brandenburg and Anderson [10]; we did not formally characterize it here. Under A, a reader holds the lock through its read, completing it before the next exclusive acquisition can finish. With a free-running writer and approximately one read per reader admission, this predicts

$$R_A \simeq 2rW_A. \tag{1}$$

Across ten trials in each of eleven non-legacy X1/X3 cells, individual ratios $R/(rW)$ ranged from 2.002 to 2.015. Legacy ratios ranged from 1.98 to 2.29, with only about 26 commits per trial at $r = 4$ and six at $r = 16$. Non-legacy coupling stayed approximately unchanged with dose or reader count.

A-min and B cell mean ratios fell from about 1.7 at zero delay to about 1.3 at 20 and 200 μs, about 1.0 at 2 ms, and 0.007–0.018 in legacy (Table III). Their I/O occurred outside the lock, so the writer's short second acquisition did not wait for it. A reader still doing I/O could miss that admission and block behind the next commit. This account was consistent with the counters and Fig. 1; the pre-registered follow-up in Section V-C tested it.

A descriptive commit-cycle check used $T = 1/W$ and A's measured mean first-acquisition wait w_A:

$$\widehat{T}_A = T_{\text{A-min}} + 2w_A. \tag{2}$$

We averaged per-trial cycles, not reciprocals of aggregate rates. Table III gives all ten X1 comparisons. Prediction error $(\widehat{T}_A/T_A - 1)$ ranged from -13.8% to $+10.6\%$. Only the first wait was instrumented, so an equal second wait was assumed. This was a ten-cell consistency check, not a validated model.

The dose also reached the writer: `latestValueLocked` read delayed SSTable blocks during conflict validation under exclusive `Mu` in every arm. This explained decreasing writes in A-min and B too. A-min's mean cycles at $r = 4$ were 1.07, 1.06, 1.23, and 2.79 ms. The denominator of S also degraded; leasing and snapshots removed only the reader's I/O hold.

Read p50 was unstable. At $r = 4, d = 0$, geometric mean trial p50 was 45 μs under A versus 808 μs under A-min, but p99 was 2.27 versus 2.31 ms. Reader cycles mixed a fast read with one blocked behind a roughly 1 ms commit. Bimodality was inferred from percentiles and counters; no full histogram was reported. Throughput and p99 were more informative than p50 alone.

C. Intervention test (pre-registered follow-up)

A separately pre-registered follow-up campaign, X-intervention-v1, tested the admission account on new data. An experiment-only flag, off by default, moved the threshold check inside the first critical section: below threshold, `inline` skipped the second acquisition; rotation and flush behavior remained unchanged.

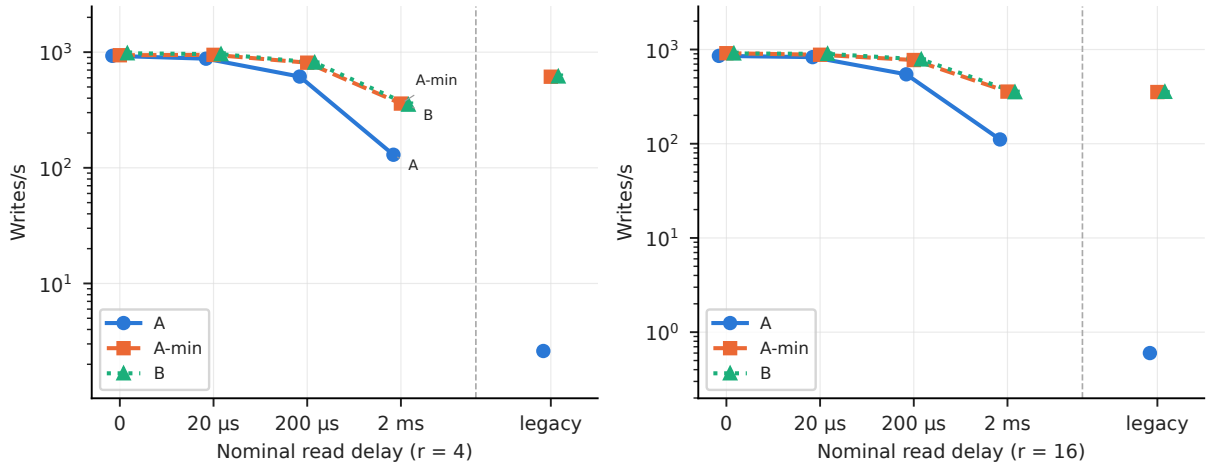


Fig. 2. X1 writes, $n = 10$ triplets per cell. Points are per-arm geometric means; bars are 95% t CIs on log throughput, exponentiated. Most intervals are smaller than the markers. Legacy is an isolated category.

TABLE II
X1, TEN TRIPLETS PER CELL. RATES: GEOMETRIC MEANS (OPERATIONS/S). CS: ARITHMETIC MEAN OF A'S TRIAL MEAN READ CRITICAL SECTIONS (μS). PAIRED RATIOS HAVE 95% t CIS.

			Writes/s				Reads/s			
	Nominal	A CS				$S = W_A/W_{\text{A-min}}$				$R_A/R_{\text{A-min}}$
r	d	μs	A	A-min	B	[95% CI]	A	A-min	B	[95% CI]
4	0	13.6	929	940	978	0.988 [0.896, 1.089]	7,468	6,258	6,482	1.193 [1.089, 1.308]
4	20 μs	33.6	878	948	961	0.926 [0.881, 0.974]	7,035	4,844	4,909	1.452 [1.380, 1.528]
4	200 μs	163	615	811	813	0.758 [0.725, 0.793]	4,932	4,154	4,161	1.187 [1.136, 1.241]
4	2 ms	1,885	130	359	352	0.361 [0.354, 0.368]	1,039	1,434	1,408	0.725 [0.710, 0.739]
4	legacy	124,853	2.6	613	617	0.004250 [0.004049, 0.004461]	21.2	43.3	43.3	0.489 [0.468, 0.511]
16	0	21.2	856	914	909	0.937 [0.884, 0.993]	27,538	25,101	24,972	1.097 [1.041, 1.156]
16	20 μs	39.3	830	880	899	0.944 [0.887, 1.004]	26,668	18,620	18,966	1.432 [1.350, 1.519]
16	200 μs	173	545	773	785	0.705 [0.676, 0.736]	17,507	16,017	16,256	1.093 [1.048, 1.140]
16	2 ms	1,915	111	358	352	0.311 [0.305, 0.316]	3,557	5,693	5,617	0.625 [0.614, 0.636]
16	legacy	430,130	0.6	354	355	0.001696 [0.001661, 0.001732]	21.2	40.6	40.7	0.522 [0.501, 0.544]

Eight X1-shaped cells ($r = 4, 16$; $d = 0$, 20 μs, 200 μs, 2 ms) compared `second` (default) and `inline` in A and A-min over five randomized rounds, sharing one seed across each four-run cell-round unit (160 trials; rates used the fixed 10 s window). The baseline was rerun in the same session; B was omitted.

P1 required A's arithmetic mean q in $[1.90, 2.10]$ under `second` and $[0.90, 1.10]$ under `inline`, in all eight cells. P2 required the exponentiated 95% paired log-t CI for the inline/second ratio of $R_A/R_{\text{A-min}}$ wholly below one in the six cells at $d \leq 200$ μs ($n = 5$, no multiplicity correction; descriptive strength only). P3 required cycle errors within $\pm 15\%$ in all sixteen condition-cells, using $T_{\text{A-min}} + 2w_A$ under `second` and $T_{\text{A-min}} + w_A$ under `inline`, plus write-ratio CIs wholly above one at 200 μs and 2 ms for both reader counts.

All 160 trials succeeded on the first attempt on 2026-11-02, with no retries or dropped units. P1 was supported in 16/16 condition-cells: individual A trial q ranged from 2.002–2.013 under `second` and 1.001–1.006 under `inline`. P2 was supported in 6/6 cells: at $d = 0$, 20, 200 μs, the ratio of read ratios was 0.823 $[0.716, 0.946]$, 0.659 $[0.595, 0.730]$, 0.755 $[0.707, 0.807]$ at $r = 4$; and 0.895 $[0.822, 0.975]$, 0.755 $[0.632, 0.903]$, 0.824 $[0.781, 0.869]$ at $r = 16$. Inline read-ratio CIs included one at $d \leq 20$ μs and lay below one at 200 μs (Table IV): A's low-cost read advantage was no longer detectable and reversed at 200 μs. P3 was supported: cycle errors ranged from -3.7% to $+7.5\%$ under `second` and -2.0% to $+8.4\%$ under `inline`; all four predicted write-ratio CIs exceeded one.

Unpredicted findings qualified these results. The second wait accounted for a substantial part of A's write penalty at 200 μs and 2 ms, but a penalty remained (Table IV, S). Inline gave both arms $q \simeq 1.00$ through 200 μs, so the read ratio essentially equalled S. Absolute reads fell: inline/second ratios were 0.507–0.763 (A) and 0.608–0.850 (A-min). A-min write ratios were 0.970–1.044, all CIs including one. Its second-condition q was 1.655–1.722 at zero delay, about 1.28–1.32 at 20/200 μs, and 1.00 at 2 ms; inline gave about 1.00 through 200 μs and 0.857–0.858 at 2 ms. The latter was not investigated. Read p99 rose in both arms: geometric mean paired inline/second trial-p99 ratios from raw trial JSON were 1.330–2.049 (A) and 1.578–2.125 (A-min).

Baseline q reproduced the main pattern, but S at $r = 16, d = 20$ μs was 0.855 versus 0.944 in the main campaign;

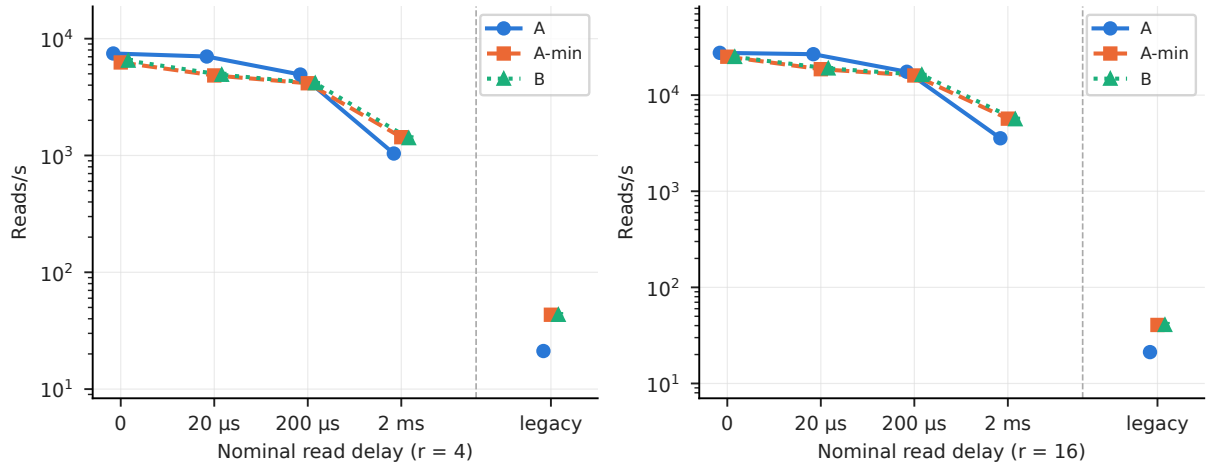


Fig. 3. X1 reads, $n = 10$ triplets per cell. Points are geometric means; bars are exponentiated 95% log-t CIs. A's low-dose read advantage was unpredicted.

TABLE III
EXPLORATORY MECHANISM: ARITHMETIC MEANS OVER TEN TRIALS PER CELL. q = IN-WINDOW READS/(r IN-WINDOW COMMITS). CYCLES ARE MS; ERROR IS $100(\widehat{T}/T - 1)$. X3 REPORTS ADMISSION RATIOS ONLY.

Block	r	Nominal d	q_A	$q_{\text{A-min}}$	q_B	T_A	$\widehat{T}_A$	Error (%)
X1	4	0	2.009	1.664	1.657	1.077	1.130	4.9
X1	4	20 μs	2.003	1.278	1.277	1.140	1.175	3.1
X1	4	200 μs	2.004	1.280	1.280	1.627	1.738	6.8
X1	4	2 ms	2.003	0.998	1.001	7.712	7.432	-3.6
X1	4	legacy	2.032	0.018	0.018	384.3	363.6	-5.4
X1	16	0	2.010	1.717	1.718	1.169	1.249	6.8
X1	16	20 μs	2.008	1.323	1.318	1.206	1.334	10.6
X1	16	200 μs	2.006	1.295	1.294	1.835	1.978	7.8
X1	16	2 ms	2.003	0.995	0.996	9.008	8.726	-3.1
X1	16	legacy	2.209	0.007	0.007	1,666.7	1,437.3	-13.8
X3	1	0	2.009	1.080	1.081	–	–	–
X3	4	0	2.009	1.100	1.099	–	–	–
X3	16	0	2.013	1.104	1.101	–	–	–

the $n = 5$ and $n = 10$ data were not pooled. The intervention supported A's read/commit coupling to acquisition count in this engine and workload. The second acquisition was not simply waste: it provided another reader admission. Removing it traded reads and read tail latency for A's writes, without a detectable A-min write increase; this is not a general optimization recommendation.

D. RQ2: A-min versus B

All ten X1 write cells and fourteen X1/X2 read cells met H2 equivalence after Holm correction (Fig. 5). All corresponding bootstrap intervals lay inside the same margin, agreeing with the throughput classifications; no bootstrap TOST replaced the planned test. Equivalence did not imply identity: X2 at $d = 0$ had read ratio 0.970, 90% CI $[0.962, 0.978]$, a detectable roughly 3% difference inside the margin.

Publication lock-wait p99 did not establish equivalence (Table V). At $d = 0$, A-min/B was 3.11, 90% CI $[1.54, 6.30]$, classified not equivalent; the other cells were inconclusive. Cell geometric mean p99 waits were below 42 μs for both arms. A p99 over twenty events was close to the maximum, making this outcome noisy. Throughput equivalence did not extend to publication lock wait.

E. X2: publication

At zero delay, 200 μs, and 2 ms, every arm published 20/20 tables within the window in every trial (Table VI, Fig. 4). In legacy, A's median was 10.5, range 9–11; A-min and B completed 20/20 throughout. Publication deadline failure therefore appeared only at legacy in this setting. A's geometric mean publication lock-wait p99 was 247 μs, 450 μs, 3.89 ms, and 1.05 s (legacy arithmetic mean 1.06 s). At zero delay, A-min and B were 41.6 and 13.4 μs. X2 A/A-min read ratios were 1.05, 1.00, 1.00, and 0.65.

F. RQ3: churn

With 16 KiB memtables, mean measured flushes ranged from 38.8 to 73.2 per trial (10 s plus drain). Table VII gives rates and paired ratios. All 95% paired-t CIs included one, but bootstrap intervals at $r = 1, 16$ narrowly excluded one for both outcomes, e.g., $[0.909, 0.997]$ for reads at $r = 1$. Small B-favoring differences could not be ruled out.

TABLE IV

INTERVENTION, $n = 5$ PAIRED FOUR-RUN UNITS PER CELL. q IS AN ARITHMETIC MEAN; W AND R RATIOS ARE PAIRED GEOMETRIC MEANS. SHOWN CIS ARE EXPONENTIATED 95% PAIRED LOG-t CIS. CONDITIONS: S = SECOND, I = INLINE. $S = W_A/W_{\text{A-min}}$.

r	Nominal d	q_A s	q_A i	$q_{\text{A-min}}$ s	$q_{\text{A-min}}$ i	$W_{A,i}/W_{A,s}$ [95% CI]	S s	S i	$R_A/R_{\text{A-min}}$ [95% CI] s	$R_A/R_{\text{A-min}}$ [95% CI] i
4	0	2.008	1.004	1.655	1.004	1.014 [0.943, 1.091]	0.965	0.963	1.170 [1.066, 1.285]	0.963 [0.897, 1.033]
4	20 μs	2.003	1.002	1.281	1.002	1.032 [0.928, 1.147]	0.946	0.974	1.479 [1.363, 1.604]	0.975 [0.882, 1.077]
4	200 μs	2.003	1.002	1.281	1.002	1.191 [1.131, 1.253]	0.731	0.864	1.144 [1.062, 1.232]	0.864 [0.758, 0.985]
4	2 ms	2.003	1.001	0.998	0.857	1.468 [1.429, 1.508]	0.356	0.539	0.715 [0.706, 0.726]	0.630 [0.592, 0.671]
16	0	2.010	1.003	1.722	1.003	1.091 [1.050, 1.134]	0.945	0.987	1.103 [1.064, 1.142]	0.987 [0.895, 1.089]
16	20 μs	2.008	1.003	1.320	1.003	1.123 [0.992, 1.271]	0.855	0.983	1.301 [1.174, 1.443]	0.983 [0.897, 1.077]
16	200 μs	2.006	1.001	1.290	1.001	1.274 [1.175, 1.382]	0.662	0.848	1.030 [0.993, 1.068]	0.848 [0.822, 0.875]
16	2 ms	2.003	1.001	0.997	0.858	1.528 [1.507, 1.548]	0.310	0.479	0.623 [0.603, 0.643]	0.559 [0.553, 0.566]

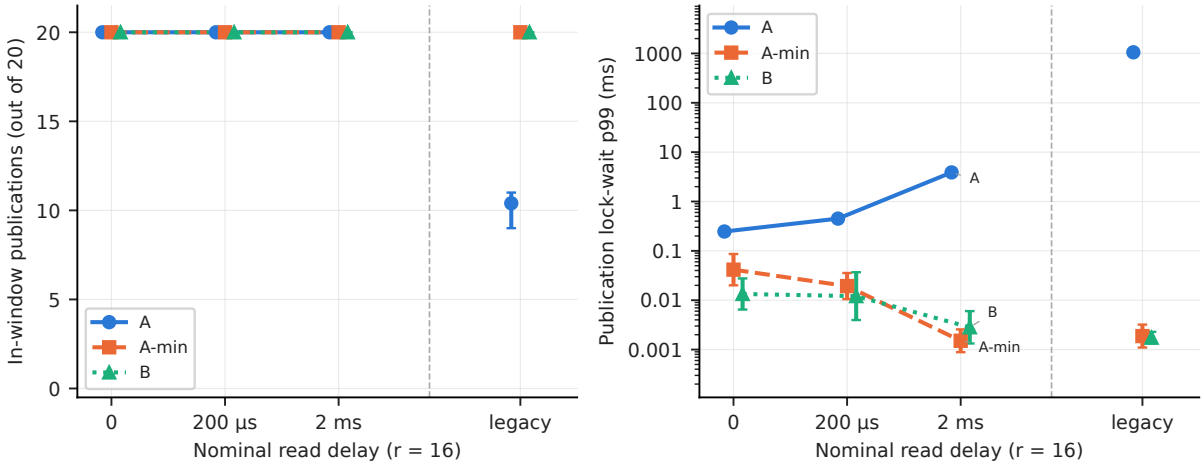


Fig. 4. X2, $n = 10$ triplets per cell. Left: arithmetic mean in-window count, min–max bars. Right: geometric mean trial publication lock-wait p99, exponentiated 95% log-t CIs. Wait instrumentation included drain.

TABLE V

H2 PUBLICATION LOCK-WAIT P99, $n = 10$ TRIPLETS: PAIRED RATIO, 90% t CI, AND PLANNED CLASSIFICATION.

Nominal d	A-min/B [90% CI]	Class
0	3.11 [1.54, 6.30]	not equivalent
200 μs	1.60 [0.51, 4.98]	inconclusive
2 ms	0.53 [0.22, 1.29]	inconclusive
legacy	1.08 [0.64, 1.82]	inconclusive

A/B read ratios were 1.66, 1.53, 1.29; write ratios were 0.89, 0.84, 0.71. The read deficit relative to A also occurred in A-min at the same size. A-min did not allocate reader-set snapshot descriptors, so these data did not support attributing the deficit to snapshot lifecycle cost or claiming A-min avoided a churn penalty. This was consistent with admission ratios near 2.01 under A and about 1.1 under both alternatives.

Pre-registered anchor drift was $D = R_{\text{A-min}}(t10)/R_{\text{A-min}}(t00) = 0.979$, $|\ln D| = 0.021$, below the 0.10 threat threshold. For transparency, A and B end/start ratios were both about 1.09, log ratios 0.087 and 0.090: below, but near, that threshold. Each anchor arm/time was a single trial.

VI. DISCUSSION

The write penalty grew as A's shared critical section lengthened and was small at tens of microseconds. The intervention supported the acquisition-count account of its low-cost read advantage: the second acquisition gave readers another admission. File lifetime and admission frequency should be evaluated together.

Leasing matched snapshots within the declared throughput margin in X1/X2. This established no advantage of A-min over B and removed neither exclusive conflict validation nor fsync. Under churn, the read deficit relative to A appeared at the same size in A-min and B, so it could not be attributed to descriptor allocation. Publication wait required a separate conclusion from throughput.

These implications are conditional on this class of engine. Measure lock-held read duration before replacing a coarse lock: in the main campaign, below about 40 μs of measured hold, point-estimate write penalties were within 8% and A completed more reads. At nominal 20 μs, the 95% CIs for S were $[0.881, 0.974]$ at $r = 4$ and $[0.887, 1.004]$ at $r = 16$, so 8% is not a confidence bound. Above about 150 μs, point-estimate penalties were at least 24%. Shortening reader holds did not remove the writer's own exclusive work, which set

TABLE VI
X2, TEN TRIPLETS: GEOMETRIC MEAN READ RATE AND TRIAL LOCK-WAIT P99 (μS); IN-WINDOW PUBLICATIONS ARE MEDIAN [MIN, MAX], OUT OF TWENTY.

Nominal	Reads/s			In-window publications			Lock-wait p99 (μs)		
d	A	A-min	B	A	A-min	B	A	A-min	B
0	318,567	304,721	314,201	20 [20, 20]	20 [20, 20]	20 [20, 20]	247	41.60	13.37
200 μs	75,018	74,678	74,844	20 [20, 20]	20 [20, 20]	20 [20, 20]	450	19.33	12.10
2 ms	8,437	8,445	8,450	20 [20, 20]	20 [20, 20]	20 [20, 20]	3,886	1.51	2.83
legacy	27.6	42.7	42.7	10.5 [9, 11]	20 [20, 20]	20 [20, 20]	1,054,655	1.88	1.74

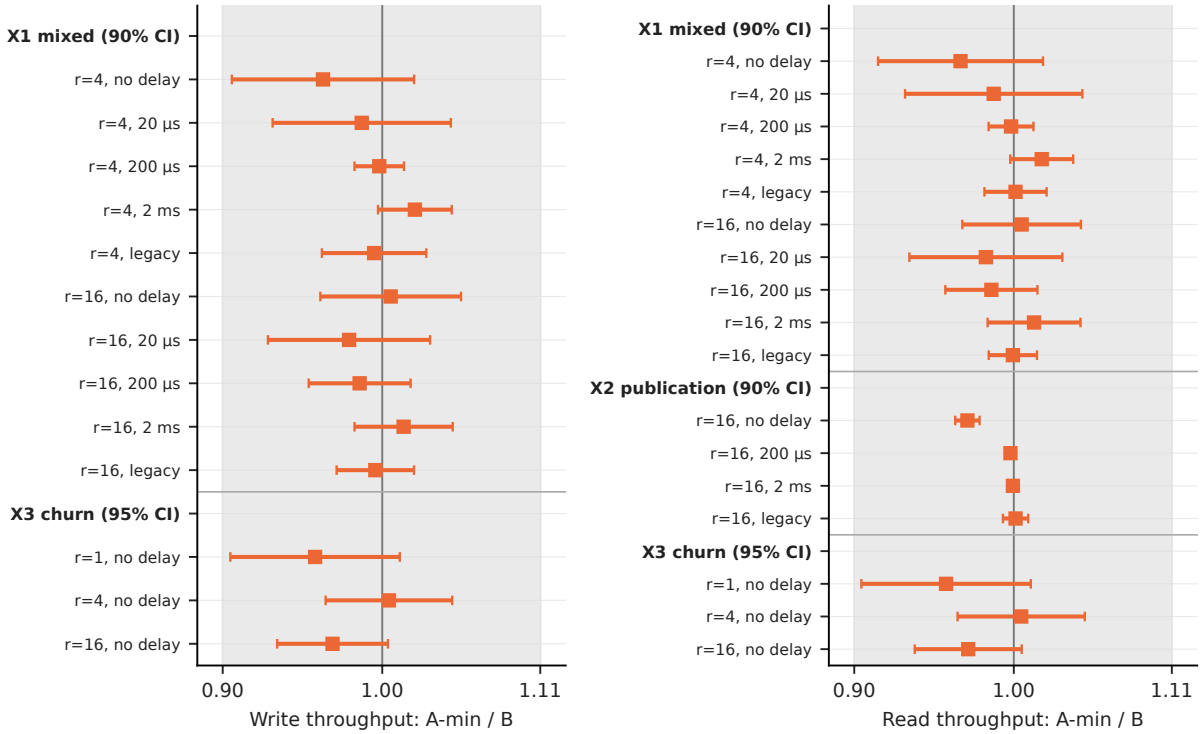


Fig. 5. A-min/B throughput, $n = 10$ triplets per cell. Points: paired geometric ratios; bars: exponentiated paired log-t CIs, 90% in X1/X2 and 95% in X3. Shading: rounded H2 margin $[0.90, 1.11]$. X3 was not an equivalence test.

the commit cycle in A-min and B. The number of exclusive acquisitions per commit changed both reader admission and A's writes, and removing one reduced reads and raised read p99, so it is not a general optimization. A lock's admission policy belongs in the performance model. We did not establish that these results transfer to other engines.

Future work includes moving fsync and conflict validation outside the exclusive lock, other lock policies, group commit, larger datasets, and multiple writers, with correctness validation as well as performance measurement.

VII. THREATS TO VALIDITY

Injected delay was a controlled proxy, not a device model. Every visited point-read SSTable block incurred delay without application block-cache hits. Sub-millisecond delays spun while yielding; sixteen readers exceeded twelve logical CPUs, so CPU contention was part of those cells. The 10,000-key dataset was likely in the OS page cache. Uniform, closed-loop access and one writer restricted workload scope.

WAL fsync remained under the exclusive lock without group commit, setting an approximately 1 ms commit floor in low-cost cells. LevelDB releases its mutex during log writing and synchronization [2]; results do not transfer directly to engines with group commit. The optional WAL-sync-off run was not performed; the X harness rejected `wal_sync=false`. Two acquisitions and Go's policy were engine-specific: Eq. 1 should not be generalized.

The mechanism was found post hoc and tested on new data with predictions frozen in git before real trials. The intervention had $n = 5$ per cell, no multiplicity correction for P2, and omitted B. Eight smoke trials were seen before the freeze and excluded; predictions were written before those trials. The intervention applies to one engine and one lock implementation. The original protocol's statement that A-min and B did not hold the lock across delay applied to

TABLE VII
X3, TEN TRIPLETS PER CELL. ARM RATES: GEOMETRIC MEANS (OPERATIONS/S). A-MIN/B: 95% PAIRED-t CI AND 95% PERCENTILE BOOTSTRAP CI. FLUSHES: ARITHMETIC MEANS FOR A/A-MIN/B, INCLUDING DRAIN.

r	Outcome	A	A-min	B	A-min/B [95% t CI]	Bootstrap CI	Flushes A/A-min/B
1	W	697	747	781	0.957 [0.905, 1.012]	[0.910, 0.997]	65.3/70.1/73.2
1	R	1,400	807	844	0.956 [0.904, 1.011]	[0.909, 0.997]	–
4	W	585	702	699	1.004 [0.963, 1.047]	[0.970, 1.038]	54.6/65.9/65.5
4	R	4,703	3,089	3,074	1.005 [0.964, 1.048]	[0.970, 1.038]	–
16	W	415	569	588	0.968 [0.933, 1.004]	[0.938, 0.995]	38.8/53.2/55.1
16	R	13,351	10,050	10,357	0.970 [0.937, 1.005]	[0.941, 0.997]	–

readers only; delayed writer conflict checks were noticed after collection. The cycle model assumed an unmeasured second wait; inline removed this assumption below threshold. Bimodality was inferred without a full histogram. One laptop, OS, and file system, with Defender enabled, limited external validity.

An operator stopped the first start at 38 of 519 trials for power-supply reasons unrelated to results. It was discarded unanalysed; only status lines were seen. A 21-trial smoke run was seen before analysis rules were fixed and was excluded; no design value changed because of it. Fixtures were built without fsync. Race detection passed on Linux under WSL2 with Go 1.27.3, not on Windows. The author was sole experimenter and analyst; there was no independent replication.

VIII. RELATED WORK

The LSM-tree [1] and survey [5] frame the organization; LevelDB and RocksDB [2], [3] are reference implementations. Dong et al. [4] describe RocksDB's evolving priorities. Our arms do not reproduce production designs. Golan-Gueta et al. [6] propose cLSM, an algorithm for scalable concurrency in LSM data stores. SILK [7] addresses LSM latency spikes.

Courtois et al. [8] pose the readers–writers problem, Mellor-Crummey and Scott [9] give scalable reader-writer algorithms, and Brandenburg and Anderson [10] propose phase-fair locks. Go's RWMutex documentation [12] supplies the policy here. Page's ordered-rank test [13], Schuirmann's TOST [14], and Holm's correction [15] supplied the pre-registered methods.

IX. CONCLUSION

Across the 519-trial campaign, coarse read locking's write penalty increased with measured critical-section length and was small at tens of microseconds; A completed more reads through nominal 200 μs. A separately pre-registered 160-trial intervention supported the admission mechanism: removing the second acquisition reduced A's reads per reader per commit from about two to one and removed its detectable low-cost read advantage. It improved A's writes at higher cost while reducing both arms' reads and increasing p99, without a detectable A-min write increase. Leasing matched snapshots within the declared throughput margin in all ten X1 write and fourteen X1/X2 read cells; publication lock-wait equivalence was not established. Neither removed writer exclusive work, and churn did not isolate snapshot lifecycle cost. Findings remain bounded by this engine, host, and workload.

DATA AVAILABILITY

Source, plan, frozen protocol, analysis and figure scripts are in https://github.com/itsramhere/selfdbms, branch `research/x-readcost`; trial records are archived with the repository. The main measured revision was `c53ee5b`, plan SHA-256 prefix `68ad96f878c1`, runner `22060ffc4885`. The intervention used `4c2749b`, plan prefix `2e44fcacb3e7`, runner `5dc57e7d85d7`, and frozen protocol `research/experiments/phase_i7/X_INTERVENTION_PREREGISTRATION.md`. Tables came from final results and trial envelopes via `x_readcost_paper_tables.py`. A claim-to-evidence map is at `paper/CLAIM_EVIDENCE_MAP_X.md`.

REFERENCES

[1] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil, "The log-structured merge-tree (LSM-tree)," *Acta Informatica*, vol. 33, no. 4, pp. 351–385, 1996.

[2] S. Ghemawat and J. Dean, "LevelDB," GitHub repository google/leveldb. [Online]. Available: https://github.com/google/leveldb

[3] Meta Platforms, "RocksDB," GitHub repository facebook/rocksdb. [Online]. Available: https://github.com/facebook/rocksdb

[4] S. Dong, A. Kryczka, Y. Jin, and M. Stumm, "RocksDB: Evolution of development priorities in a key-value store serving large-scale applications," *ACM Trans. Storage*, vol. 17, no. 4, Art. 26, 2021.

[5] C. Luo and M. J. Carey, "LSM-based storage techniques: a survey," *The VLDB Journal*, vol. 29, no. 1, pp. 393–418, 2020.

[6] G. Golan-Gueta, E. Bortnikov, E. Hillel, and I. Keidar, "Scaling concurrent log-structured data stores," in *Proc. EuroSys*, 2015, Art. 32.

[7] O. Balmau, F. Dinu, W. Zwaenepoel, K. Gupta, R. Chandhiramoorthi, and D. Didona, "SILK: Preventing latency spikes in log-structured merge key-value stores," in *Proc. USENIX ATC*, 2019, pp. 753–766.

[8] P. J. Courtois, F. Heymans, and D. L. Parnas, "Concurrent control with 'readers' and 'writers'," *Commun. ACM*, vol. 14, no. 10, pp. 667–668, 1971.

[9] J. M. Mellor-Crummey and M. L. Scott, "Scalable reader-writer synchronization for shared-memory multiprocessors," in *Proc. PPoPP*, 1991, pp. 106–113.

[10] B. B. Brandenburg and J. H. Anderson, "Reader-writer synchronization for shared-memory multiprocessor real-time systems," in *Proc. ECRTS*, 2009, pp. 184–193.

[11] W. Pugh, "Skip lists: A probabilistic alternative to balanced trees," *Commun. ACM*, vol. 33, no. 6, pp. 668–676, 1990.

[12] The Go Authors, "sync package: RWMutex." [Online]. Available: https://pkg.go.dev/sync#RWMutex

[13] E. B. Page, "Ordered hypotheses for multiple treatments: A significance test for linear ranks," *J. Amer. Statist. Assoc.*, vol. 58, no. 301, pp. 216–230, 1963.

[14] D. J. Schuirmann, "A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability," *J. Pharmacokinet. Biopharm.*, vol. 15, no. 6, pp. 657–680, 1987.

[15] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.